

Retreinamento Reativo em Aprendizado Federado sob Drift Gradual: Detecção Local por Incerteza e Decisão Coletiva por Quórum

Julio Amaral*, Hugo César[†], Miguel Elias M. Campista[†], Luiz Fernando Bittencourt*

*Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Brasil
j230537@dac.unicamp.br, bit@unicamp.br

[†]Universidade Federal do Rio de Janeiro (UFRJ), Brasil
hugocesar@gta.ufrj.br, miguel@gta.ufrj.br

Resumo—Modelos de aprendizado federado (FL) implantados em ambientes reais enfrentam concept drift: a distribuição dos dados muda após a implantação, e a qualidade do modelo se deteriora até que uma ação de retreinamento seja adotada. Em implantações reais a mudança nem sempre é abrupta: a fração de entradas degradadas cresce lentamente ao longo de um intervalo de transição cuja duração é a largura do drift. Estudamos o retreinamento reativo em FL síncrono à medida que a transição do drift se alonga do regime abrupto ao gradual. Cada cliente monitora seu fluxo de entrada não rotulado com um detector local baseado em incerteza, e o servidor agrega os alarmes ao longo de uma curta janela de passos para disparar uma rodada de retreinamento sobre os dados correntes de cada cliente. A métrica primária é o downtime de serviço, o tempo acumulado em que a acurácia do modelo em serviço permanece abaixo de um limiar de qualidade. Em experimentos pareados contra uma baseline sem retreinamento, a política reativa detecta quase todos os episódios e reduz de forma significativa o downtime, anulando-o no melhor caso; rampas longas também preservam melhor a acurácia no conjunto limpo. O gatilho coletivo é disparado sob drift—inclusive staggered—oque não ocorre em fluxos sem drift, enquanto o ruído Gaussiano se mostra o cenário mais difícil, com a maior latência de detecção. No regime exercitado, uma política reativa sem rótulos é eficaz, e a latência de detecção depende do cenário.

Palavras-chave—aprendizado federado, concept drift, detecção de drift, retreinamento reativo, detecção não supervisionada, downtime de serviço.

I. Introdução

Aprendizado federado (FL) treina um modelo compartilhado a partir de dados que permanecem distribuídos entre os clientes [1]. Uma vez implantado, o modelo é servido em um ambiente cuja distribuição de dados pode mudar; tal concept drift compromete a qualidade do modelo em serviço até que uma ação de retreinamento seja adotada, e detectá-lo—em geral sem rótulos no momento da inferência—é um pré-requisito para dispará-la. A mudança nem sempre é abrupta: no drift gradual, dados novos passam a coexistir com os antigos, e a proporção de entradas deslocadas cresce lentamente ao longo de um intervalo de transição cuja duração é a largura do drift usada na literatura de detecção de drift [3], [18]. Modelamos esse contínuo como uma rampa linear da fração de dados degradados de zero a um ao longo de

um número configurável de passos de monitoramento [3]. Nossa grade abrange rampas curtas (o regime abrupto) e uma rampa longa na qual o detector opera em meio à transição.

Nossa política reativa segue a arquitetura predominante na literatura de drift em FL: detecção local, decisão central [2], [4]–[10], [24]. Cada cliente executa um detector de drift por incerteza (UDD) [11]: Monte Carlo (MC) dropout [12] ao longo de T forward passes estocásticos, entropia média de Shannon e um teste de janela adaptativa ADWIN [13] por cliente. O servidor coleta os alarmes por cliente ao longo de uma janela deslizante de dois passos de monitoramento e dispara quando a fração de clientes distintos sinalizados atinge um quórum. A ação disparada é uma única rodada global de retreinamento sobre os dados que cada cliente serve no momento, em consonância com os orçamentos explícitos de retreinamento do DriftGuard [8] e do ShiftEx [26] e com a separação entre gatilho e seleção de dados do Modyn [20].

A métrica primária é o downtime de serviço: o tempo acumulado em que a acurácia do modelo em serviço sob degradação permanece abaixo de um limiar de qualidade τ . Até onde sabemos, nenhum trabalho na literatura revisada de drift em FL quantifica o tempo em que um modelo em serviço permanece abaixo de um limiar de qualidade; detalhamos essa lacuna na Seção II-E.

Nossas contribuições são: (i) uma métrica de downtime de serviço para FL sob drift, calculada como integral em escada sobre um horizonte fixo; (ii) uma política coletiva por quórum que agrega alarmes sem rótulos por cliente ao longo de uma curta janela de passos, ausente na literatura revisada e mais próxima da barreira de participação do FL-MalDrift [7], cuja latência de detecção caracterizamos por meio de ablações de janela e sob staggered drift; (iii) uma implementação federada do UDD, com um detector por cliente compartilhando o modelo global, estendendo o detector centralizado de Baier et al. [11]; (iv) uma infraestrutura experimental auditável que parecia a variante reativa com uma baseline sem retreinamento a partir do mesmo checkpoint, estado aleatório, fluxo e horizonte; e (v) uma caracterização empírica do drift gradual ao longo

de 60 episódios pareados.

II. Contextualização e Trabalhos Relacionados

A. Concept drift em aprendizado federado

Concept drift é a mudança na distribuição dos dados ao longo do tempo [19], [21]; a taxonomia distingue drift virtual (só $P(X)$ muda) de drift real (a condicional $P(Y | X)$ muda), bem como drifts abruptos, graduais e recorrentes [19]. Staggered drifts, nos quais clientes diferentes sofrem drift em momentos diferentes, são estudados por Jothimurugesan et al. [4]. O drift gradual, regime estudado aqui, combina o conceito novo com o antigo em proporção crescente ao longo do tempo [2]; a duração da transição é a largura do drift [3], [18].

Nosso escopo limita-se a drifts acompanhados de mudança em $P(X)$ (covariate shift, com rótulos preservados). É o pressuposto sob o qual a detecção sem rótulos é estruturalmente possível: métodos sem rótulos são funções determinísticas das entradas, de modo que uma mudança restrita a $P(Y | X)$ não produz sinal observável — um ponto cego medido por Solozobov [17] e declarado por Baier et al. [11].

B. Detecção local, decisão central

Um achado recorrente é que a detecção de drift precisa ser local. Jothimurugesan et al. [4] mostram que testar o erro agregado no servidor falha durante transições staggered, porque os erros de clientes que já sofreram drift e dos que ainda não sofreram se cancelam. FLARE [5], FLAME [6], FL-MalDrift [7], DriftGuard [8], FedDAA [9], FedPLC [10], FELK/FL-HVD [24] e Casado et al. [2] seguem detecção local como decisão central. Em outros trabalhos: o FLASH detecta drift no servidor a partir da magnitude das atualizações agregadas de parâmetros [23]; Stallmann et al. usam um detector global por agrupamento fuzzy [25]; e o Adaptive-FedAVG dispensa por completo a detecção, adaptando a taxa de aprendizado no servidor para aumentar a plasticidade da fase de aprendizado [27]. Nossa arquitetura segue esse padrão predominante.

C. Detecção de drift sem rótulos

O detector de drift por incerteza (UDD) de Baier et al. [11] é o método de referência sem rótulos; a Seção IV-A descreve nossa implementação. O ADWIN é preferido a DDM/EDDM porque estes exigem entradas binomiais, enquanto o sinal de entropia não se restringe a esse tipo [11]. Lobo et al. [14] conectam concept drift e incerteza do modelo, e sabe-se que estimadores de incerteza perdem confiabilidade sob dataset shift [15]. O D3M [16] é um método complementar sem rótulos, baseado em discordância entre modelos, com garantias quanto a falsos positivos sob o pressuposto de covariate shift. O PUDD [28] é igualmente complementar: aplica um teste qui-quadrado de Pearson a um índice de incerteza de predição, cujo nível de significância controla a taxa de falsos positivos — ao contrário da etapa ADWIN do UDD, que acompanha a média.

D. Orquestração e orçamentos de retreinamento

O DriftGuard [8] formaliza a decisão de retreinamento como uma tupla $\pi^t = (\text{Trig}, S, \theta)$: se deve retreinar, quais dispositivos devem participar e quais parâmetros devem ser atualizados, e relata reduções de custo de retreinamento de 80–83%. O Modyn [20] separa políticas de gatilho (tempo, volume, desempenho, drift) de políticas de seleção de dados; seu gatilho de drift usa MMD (com KS entre as estatísticas candidatas) e um limiar adaptativo por percentil. Nos trabalhos revisados de FL, só o DriftGuard [8] e o ShiftEx [26] explicitam o orçamento de retreinamento (uma quantidade nomeada que controla o número de rodadas pós-gatilho); adotamos um orçamento de uma rodada sobre a distribuição que cada cliente serve no momento.

E. Lacuna de métricas

O benchmark de detecção de drift de Cerqueira et al. [3] documenta a falta de métricas temporais padronizadas (atraso normalizado de detecção, taxa de alarmes, janelas de aceitação), o que também registram levantamentos de drift em FL [19]. Latência de detecção [5], [11], duração do retreinamento e custo por retreinamento [8] são relatados separadamente; nenhum dos trabalhos que revisamos quantifica, em uma métrica de downtime, o tempo em que o modelo em serviço permanece abaixo de um limiar de qualidade, à semelhança da prática de SLO em MLOps. A linguagem de downtime existe em trabalhos adjacentes de sistemas de ML—o desaprendizado (unlearning) federado mede o downtime de serviço como o tempo de inatividade incorrido ao retreinar para apagar dados [29]—uma noção de disponibilidade (o serviço está fora de operação), distinta do nosso tempo de operação abaixo do limiar de qualidade.

III. Definição do Problema

Consideramos FL síncrono com C clientes e tempo v , o relógio abstrato do nosso simulador. Um modelo é treinado em dados limpos durante R rodadas e entra em operação no instante v_0 . Os dados de inferência chegam em passos de monitoramento de Δt segundos: um lote sem rótulos por cliente, a cada passo.

Modelo de drift gradual. Uma degradação aplicada às entradas X (rótulos preservados) é injetada com uma fração degradada $f(v)$ que cresce linearmente de zero a um ao longo de N passos:

$$f(v) = \min\left(1, \frac{v - v_0}{N \Delta t}\right), \quad v \geq v_0. \quad (1)$$

A duração da rampa $N \Delta t$ é a largura do drift; sua inclinação $1/N$ por passo é a variável independente. Aqui $N \in \{5, 10, 20\}$ (50, 100, 200 s com $\Delta t = 10$ s).

Limiar de qualidade e downtime. Seja $a(v)$ a acurácia do modelo em serviço em um conjunto de teste com a fração corrente $f(v)$. O limiar de qualidade τ define o estado do

serviço: o modelo está indisponível quando $a(v) < \tau$. O downtime ao longo de um horizonte fixo $[v_0, v_0 + H]$ é

$$D = \sum_{i=1}^m (v_{i+1} - v_i) \cdot \mathbf{1}[a(v_i) < \tau], \quad (2)$$

em que $v_1, \dots, v_m$ são os instantes de avaliação e $v_{m+1} = v_0 + H$: a integral em escada usa a última qualidade conhecida, sem interpolação, e inclui o intervalo final até o horizonte. Os instantes de avaliação compreendem fronteiras dos passos, o instante de conclusão do retreinamento e o horizonte.

Avaliação pareada. A política reativa é comparada com uma baseline que observa o mesmo fluxo a partir do mesmo checkpoint e nunca retreina. Ambas as variantes compartilham o mesmo estado aleatório, início da operação e horizonte H ; a baseline registra, como contrafactual, o momento em que a política teria disparado.

IV. Método Proposto

A. Detecção local por incerteza (UDD por cliente)

Cada cliente c executa um detector local sobre o modelo global compartilhado: (1) o modelo é posto em modo de avaliação com as camadas de dropout reativadas; (2) $T = 5$ forward passes estocásticos produzem probabilidades por classe; (3) calcula-se a entropia média de Shannon $H = -\sum_k p_k \log p_k$ sobre o lote; (4) o escalar alimenta um ADWIN por cliente com $\delta = 0,002$ [11], [13]. Nossa implementação do ADWIN testa janelas de ao menos 20 observações, porque alimentamos o ADWIN com a entropia média de cada lote, uma vez por passo em cada cliente, e não com um fluxo de instâncias. Cada detector recebe 20 passos de warm-up em dados limpos antes do início da operação, calibrando-o sem alimentar a política coletiva.

B. Política coletiva por quórum

A cada passo de operação, o servidor recebe as flags de alarme por cliente e mantém uma janela deslizante dos últimos dois passos. A fração sinalizada é o número de clientes distintos sinalizados na janela dividido pelo número total de clientes. Quando a fração atinge o quórum $\theta = 0,30$ (3 de 10 clientes), o servidor dispara. Dessa forma, um possível falso positivo local sob monitoramento contínuo [18] não basta para iniciar o retreinamento.

C. Retreinamento reativo sobre os dados correntes

Quando a política dispara no instante t_d , o conjunto de treinamento de cada cliente é substituído por um conjunto com fração degradada $f(t_d)$ e uma única rodada síncrona de FL é executada com todos os clientes. Cada cliente detém 5 000 imagens de treinamento (uma divisão IID do conjunto de treinamento de 50 000 imagens) com fração degradada $f(t_d)$. Em todo episódio detectado a acurácia na conclusão da rodada já atinge τ , de modo que o tempo até a recuperação equivale à duração do retreinamento.

Tabela I
Calibração de severidade a partir do checkpoint limpo (intervalo estrito (0,25; 0,50)).

Degradação	Severidade	Acurácia
Ruído Gaussiano	3	0,3783
Frosted-glass blur	4	0,4022
Motion blur	1	0,3493
Fog	4	0,4875

D. Avaliação sobre os dados correntes

A série de acurácia é medida no conjunto de teste completo (10 000 imagens) com a fração corrente $f(v)$: no início da operação o teste tem fração zero, e só se torna totalmente degradado quando a rampa termina; um teste totalmente degradado superestimaria a queda de acurácia durante a rampa. A composição usa uma permutação determinística com seed por episódio, de modo que ambas as variantes são avaliadas sobre o tensor idêntico em cada instante.

E. Calibração de severidade

Todas as severidades de 1–5 são avaliadas no conjunto de teste degradado, e a menor severidade cuja acurácia cai estritamente dentro de (0,25; 0,50) é selecionada. O limite inferior mantém o modelo bem acima do acaso (0,10 para dez classes), de modo que a recuperação é alcançável com pouco orçamento de retreino; o limite superior garante que a degradação empurre o serviço para abaixo de τ .

V. Configuração Experimental

A. Conjunto de dados e degradações

Usamos CIFAR-10 com partição IID entre 10 clientes e as 10 classes completas em todas as fases. Quatro degradações visuais são implementadas diretamente sobre tensores—ruído Gaussiano, frosted-glass blur, motion blur e fog—com severidades inteiras de 1–5, geradores por seed e formas e rótulos preservados. Elas são inspiradas no benchmark CIFAR-10-C [22]; o ruído Gaussiano adiciona perturbações com desvios-padrão por severidade de 0,08–0,24; o frosted-glass blur permuta pixels em uma vizinhança 3×3 e combina-os com um borrão Gaussiano (força 0,2–1,0); o motion blur faz convolução com um kernel diagonal de largura 5–15; o fog combina a imagem com um mapa de ruído fractal de quatro oitavas.

B. Resultado da calibração

A calibração, executada uma vez a partir de um checkpoint limpo, selecionou as severidades da Tabela I (procedimento na Seção IV-E).

C. Grade

A grade do estudo é o produto cartesiano das quatro degradações calibradas, cinco seeds (42–46) e três durações de rampa (5, 10, 20 passos), totalizando 60 episódios pareados com horizonte de operação fixo de $H = 600$ s. A Tabela II lista a configuração fixa.

Tabela II
Configuração fixa de cada episódio.

Parâmetro	Valor
Conjunto de dados	CIFAR-10, IID, 10 clientes
Rodadas iniciais (limpas)	20
Passos de warm-up (limpos)	20
Duração do passo	10s
Tamanho de lote / épocas	32 / 1
Modelo	CNN 2,2M (4 conv, 2 fc)
Otimizador	Adam, lr 10^{-3}
Velocidade / timeout	uniforme (p75; rodada ≈ 8 s)
Detector	ADWIN $\delta = 0,002$; MC dropout $T = 5$
Quórum / janela	0,30 / 2 passos
Orçamento de retreinamento	1 rodada
Limiar de qualidade τ	0,50
Horizonte de operação	600s
seeds	42–46

Tabela III

Atraso de detecção (s) e progresso do drift no disparo (médias por grupo sobre 5 seeds; taxa de detecção por grupo; a seed Gaussiana que falhou em 100s está excluída da média de atraso desse grupo).

Degradação	Atraso (s)			Progresso do drift no disparo	Taxa de det.
	50 s	100 s	200 s	200 s	(de 5)
Fog	100	112	138	0.69	5/5
Frosted-glass	100	108	136	0.68	5/5
Motion blur	98	112	140	0.70	5/5
Ruído Gaussiano	164	158*	256	0.97	4/5*

Desvio-padrão do atraso entre seeds: ≤ 6 s para as degradações visuais; 71–104s para o ruído Gaussiano. Para as rampas de 50 e 100s o disparo ocorre após o fim da rampa, de modo que o progresso do drift no disparo é 1,00 por construção.

VI. Resultados

Relatamos médias por grupo sobre as cinco seeds e testamos duas hipóteses: H1, a política reativa reduz o downtime em relação à baseline sem retreinamento; H2, o atraso de detecção cresce com a duração da rampa. As medidas primárias são o atraso de detecção, o progresso do drift no disparo e o downtime; as secundárias são a clean retention (acurácia limpa final menos a acurácia limpa antes do drift) e a acurácia final sob degradação.

A. Detecção: 59 de 60 episódios

A detecção teve êxito em 59 de 60 episódios (14 de 15 no cenário de ruído Gaussiano); a única falha ocorreu no ruído Gaussiano, na rampa de 100s, com a seed 42. O atraso cresce com a duração da rampa para as três degradações visuais (Tabela III), em consonância com a análise do ADWIN para mudança gradual com inclinação constante [13]; o ruído Gaussiano não segue a tendência.

B. Downtime: política reativa domina a baseline

A política reativa reduz o downtime médio nos 12 grupos (Tabela IV), de 408–562s para 0–178s, com 59 vitórias nos 60 episódios pareados. O cenário de fog com rampa de 200s atinge downtime zero com $\tau = 0,5$ (Fig. 1): o disparo ocorre com o progresso do drift em 0,69, quando a acurácia ainda é de 0,568 (acima de τ); a rodada única mantém a acurácia acima de τ (acurácia final sob degradação de 0,71).

Tabela IV

Downtime (s): variante reativa versus baseline pareada sem retreinamento (médias por grupo sobre 5 seeds; formato: reativo / baseline).

Degradação	Duração da rampa		
	50s	100s	200s
Fog	58 / 550	24 / 504	0 / 408
Frosted-glass	68 / 560	40 / 524	3 / 454
Motion blur	68 / 562	50 / 530	14 / 466
Ruído Gaussiano	130 / 558	178 / 526	112 / 448

A dispersão do ruído Gaussiano é grande (desvio-padrão do atraso de até 104s; desvio-padrão do downtime de até 184s, intervalo 38–540s); degradações visuais têm desvio-padrão ≤ 6 s (atraso) e ≤ 10 s (downtime).

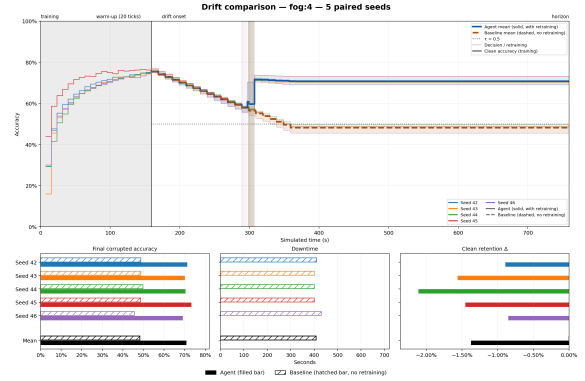


Fig. 1. Fog, rampa de 200s, cinco seeds pareadas. Acima: acurácia sob degradação ao longo do tempo simulado (sólida: reativo, tracejada: baseline; linhas grossas: médias); $\tau = 0,5$ tracejada; faixas sombreadas: a rodada de retreinamento. Abaixo: acurácia final sob degradação, downtime e clean retention; a variante reativa nunca cai abaixo de τ (downtime zero).

O contraste entre a variante reativa e a baseline mantém a mesma ordem ao variar o limiar de qualidade: para $\tau \in \{0,40; 0,45; 0,50; 0,55\}$ a variante reativa nunca supera sua baseline pareada em nenhuma das 48 células, embora as magnitudes sejam específicas de τ (o contraste médio encolhe cerca de 47% com $\tau = 0,40$; o pior caso da variante reativa cresce para 190s com $\tau = 0,55$).

C. Análise do quórum coletivo

Todos os 10 clientes emitem alarme em cada episódio detectado (59 de 59): os detectores locais são independentes. O disparo ocorre no mínimo do quórum (0,30, três clientes distintos na janela de dois passos) em 21 de 59 episódios; o ruído Gaussiano dispara no mínimo em todos os episódios detectados da rampa de 100s. A rampa longa espalha os alarmes: a fração sinalizada na decisão cai com a duração da rampa (fog 0,88 $\rightarrow$ 0,42, frosted-glass 0,90 $\rightarrow$ 0,56, motion blur 0,66 $\rightarrow$ 0,54).

Sensibilidade a quórum e janela. Com quórum 0,50, a política dispara em 59 de 60 episódios, deixando de detectar a mesma seed de ruído Gaussiano que com o

Tabela V
Clean retention Δ (variante reativa; média por grupo $\pm$ desvio-padrão sobre 5 seeds).

Degradação	Duração da rampa		
	50 s	100 s	200 s
Fog	$-0,096 \pm 0,012$	$-0,094 \pm 0,014$	$-0,014 \pm 0,005$
Frosted-glass	$-0,152 \pm 0,008$	$-0,156 \pm 0,014$	$-0,022 \pm 0,005$
Motion blur	$-0,199 \pm 0,017$	$-0,208 \pm 0,020$	$-0,016 \pm 0,002$
Ruído Gaussiano	$-0,084 \pm 0,013$	$-0,066 \pm 0,034$	$-0,072 \pm 0,016$

quórum padrão 0,30 (em rampa diferente). No quórum padrão, uma janela de um passo dá resultado neutro no cômputo geral (+3,7s de atraso, -3,9s de downtime), com o sinal da diferença variando por grupo; a união de dois passos não mostra benefício sistemático.

Staggered drift. Quando os clientes sofrem drift em passos diferentes (cliente i sofre drift no passo de operação i , sem rampa; 20 episódios), o gatilho coletivo dispara em 20 de 20 episódios no limiar do quórum (fração sinalizada 0,30-0,40). A política evita cerca de 81% do downtime da baseline (600s, drift permanente) em três das quatro degradações e 70% no ruído Gaussiano (116s e 180s de downtime na variante reativa, respectivamente); como o drift é instantâneo e permanente, atraso converte-se em downtime um para um ($116 \approx 108 + 8$ s).

D. Taxa de falsos positivos em fluxos sem drift

Executamos 15 episódios de controle (cinco seeds vezes três cronogramas de rampa) sem degradação: cada cliente monitora um fluxo limpo pelo horizonte completo de 600s. O gatilho coletivo nunca dispara—nenhum dos 15 episódios registra uma decisão de retreinamento—e os detectores locais emitem zero alarmes nas 9000 avaliações cliente-passo. Ocorrem oito picos isolados de entropia (valores acima de 1,0), mas o ADWIN não sinaliza nenhum, pois o teste é de mudança na distribuição, não de picos. O gatilho é, assim, silencioso em fluxos sem drift e sensível sob drift (59 de 60 episódios; taxa de falsos positivos medida de 0,0%).

E. Clean retention e comportamento de recuperação

Primeiro, rampas longas preservam a acurácia no conjunto limpo (Tabela V): com rampa de 200s o conjunto de retreinamento mantém cerca de 30% de dados limpos e a perda de acurácia limpa cai para 1-2 pontos (contra 9-21 em rampas curtas) para fog, frosted-glass e motion blur; o ruído Gaussiano não se beneficia (progresso do drift de 0,97 no disparo). Segundo, a recuperação não é temporária: nenhum episódio volta a cruzar τ dentro do horizonte; o modelo retreinado generaliza para o regime totalmente degradado. Em 9 dos 20 episódios com rampa de 200s (todas as cinco execuções de fog, três de frosted-glass e uma de motion blur), a acurácia nunca cai abaixo de τ . A acurácia final média por grupo, no regime totalmente degradado, varia de 0,60 a 0,71 para a variante

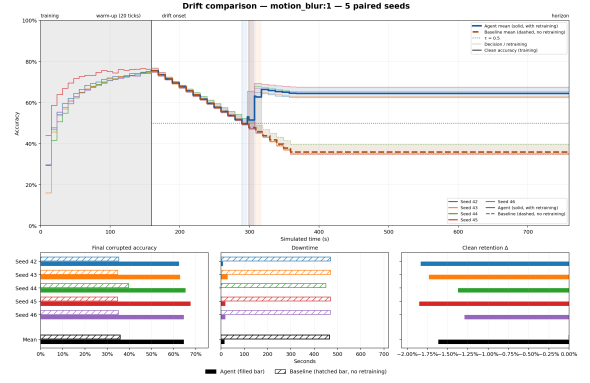


Fig. 2. Motion blur, rampa de 200 s, cinco seeds pareadas. A variante reativa recupera-se em uma rodada e permanece acima de τ durante a maior parte do horizonte; a baseline segue caindo até cerca de 0,35.

reativa (exceto no episódio com falha, cuja acurácia final é 0,33) e de 0,35 a 0,48 para a baseline (Fig. 2).

VII. Discussão

O monitoramento não é gratuito: ao longo do episódio de 80 passos (60 de operação mais 20 de warm-up), o detector com $T = 5$ custa cerca de 6,8 TFLOPs (4000 forward passes em lote de 32, 50 por passo nos dez clientes) contra cerca de 8,0 TFLOPs da rodada única de retreinamento. A razão é de 0,85-0,96 conforme a contabilização, ou seja, o monitoramento responde por cerca de 47% da computação reativa. Nos dez clientes, o monitor por passo ocupa cerca de 0,13s, 1,3% de cada passo de 10s; no dispositivo medido uma atualização do detector leva cerca de 12,7ms por cliente. Se tivéssemos mantido a configuração publicada do UDD com $T = 50$, o monitoramento excederia a rodada de retreinamento em cerca de 9 $\times$, razão pela qual a adoção de $T = 5$ foi uma decisão de custo.

As conclusões sobre o downtime são robustas a uma rodada 10 $\times$ mais longa: ao recalculer-se o downtime a posteriori a partir das séries registradas—atribuindo a acurácia pré-retreinamento ao intervalo $[t_d, t_d + d]$, usando o primeiro registro pós-retreinamento em diante e aplicando a regra de orçamento—, verifica-se que uma rodada de 80s ainda corta o downtime médio de 507,5s (baseline) para 122,2s (76%). A ordenação por grupo, o caso de downtime zero no fog e o comportamento limitado pela detecção no ruído Gaussiano permanecem inalterados; o custo extra é exatamente o intervalo pré-retreinamento que a rodada alonga. Com $d = 800$ s, a rodada já não cabe no horizonte de operação de 600s e a regra de orçamento pularia o retreinamento em todos os 59 episódios em que houve disparo, de modo que a política reativa passaria a equivaler à baseline por força da regra de orçamento, e não por dinâmica de treinamento; o ponto de virada é a menor folga entre disparo e horizonte, cerca de 140s nesta grade.

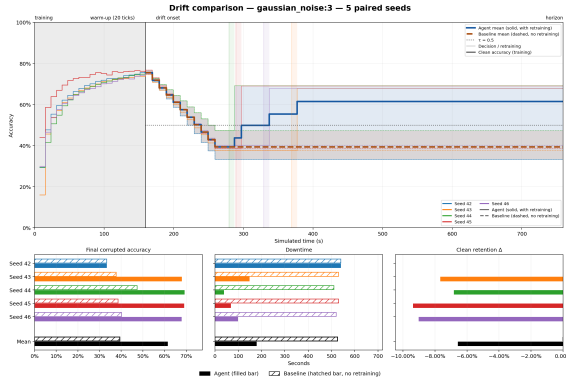


Fig. 3. Ruído Gaussiano, rampa de 100s, cinco seeds pareadas. A seed 42 nunca dispara: sua trajetória (azul sólida) não se recupera, enquanto as outras quatro seeds se recuperam para cerca de 0,68.

A. Ruído Gaussiano é o cenário limitante

O ruído Gaussiano é o único cenário com detecção incompleta. Com rampa de 100s e seed 42, a política nunca dispara: só quatro dos dez clientes emitem alarme em todo o episódio, nunca três dentro da mesma janela de dois passos (fração máxima sinalizada de 0,20 em passos esparsos); as outras quatro seeds detectam em 120–210s. Como a janela e o quórum são idênticos entre seeds, a falha é mais bem explicada como perda do detector por cliente. Sem decisão, a variante reativa equivale exatamente à baseline (downtime de 540s). Com rampa de 200s os atrasos variam entre 180 e 460s (desvio-padrão de 104s), mas a variante reativa ainda corta o downtime da baseline em 75% (112 vs. 448s), em consonância com o risco que detectores do tipo ADWIN carregam sob drifts longos e suaves [18]. O ruído Gaussiano é também o cenário de maior latência reativa. A Fig. 3 mostra a seed que falhou.

B. Por que a rampa longa se comporta melhor

Para as degradações visuais, a rampa de 200s tem três efeitos favoráveis. (i) O disparo ocorre no meio da rampa, com progresso do drift de 0,68–0,70, em que a acurácia na decisão é 0,57 para fog (ainda acima de τ), 0,51 para frosted-glass e 0,48 para motion blur. (ii) O conjunto de retreinamento contém cerca de 30% de dados limpos, reduzindo o esquecimento catastrófico. (iii) O modelo retreinado com progresso do drift de $\approx 0,7$ generaliza para o regime totalmente degradado, de modo que a recuperação se sustenta. Rampas curtas comportam-se como o regime abrupto: a detecção termina após a rampa e o conjunto de retreinamento fica quase totalmente degradado. O ruído Gaussiano dispara mais tarde (progresso 0,97) e não usufrui dos efeitos (i) e (ii).

C. Limitações

Nossos resultados são limitados pelas seguintes restrições declaradas. (i) Cinco seeds por grupo limitam a precisão da estimativa de dispersão. (ii) O modelo é uma CNN pequena de cerca de 2,2 milhões de parâmetros; as

acurácias absolutas sob degradação são modestas, como esperado para arquiteturas pequenas [22]. (iii) O cenário pressupõe um único drift, uma rodada de retreinamento, partição IID, velocidade uniforme e horizonte fixo. (iv) O progresso do drift no disparo só é informativo na rampa de 200s. (v) O UDD publicado usa $T = 50$ forward passes; o nosso $T = 5$ reduz o orçamento de monitoramento por uma questão de custo, e a comparação direta entre as duas configurações não foi investigada aqui, permanecendo como trabalho futuro; $\delta = 0,002$ é o valor padrão da implementação de referência, e não um valor calibrado [11], [18]. (vi) O downtime é amostrado na granularidade do passo mais o instante de conclusão da rodada. (vii) A proteção contra falsos positivos que o quórum oferece não pôde ser quantificada, de modo que nossas afirmações restringem-se à sua viabilidade e à taxa nula de falsos positivos em fluxos sem drift. Comparadores com agendamento, do tipo oráculo e com detector centralizado permanecem como trabalhos futuros. (viii) Episódios de controle sem drift (15) não produziram nenhum disparo falso (0/15), de modo que a taxa de falsos positivos do detector sob monitoramento contínuo é de 0% nesta configuração.

VIII. Conclusão e Trabalhos Futuros

Estudamos o retreinamento reativo em aprendizado federado síncrono ao longo do espectro do drift abrupto ao gradual, avaliando nossa política reativa contra uma baseline pareada sem retreinamento em 60 episódios, com downtime de serviço como métrica primária. A detecção teve êxito em 59 de 60 episódios, com o atraso crescendo com a duração da rampa para as degradações visuais; a variante reativa reduziu o downtime médio de 408–562s para 0–178s (zero no caso de fog com rampa de 200s); e rampas longas preservaram a acurácia limpa para as degradações visuais. O gatilho coletivo dispara nos 20 episódios de staggered drift (70–81% do downtime da baseline evitado) com taxa de falsos positivos de 0% em 15 controles sem drift. Trabalhos futuros incluem baselines com detector centralizado, do tipo oráculo e clientes do agendamento; mais seeds e análise formal de dispersão; drift recorrente; partições não IID; drift incremental; modelos maiores; sensibilidade aos parâmetros T , δ e limites de calibração do detector.

Disponibilidade

A implementação e todos os artefatos—os 60 episódios pareados, variantes comparadoras, resumos agregados e tabelas por seed—estão disponíveis em <https://github.com/julio-amaral-oliveira/FederatedLearningSimulation>.

Referências

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. AISTATS, PMLR 54, pp. 1273–1282, 2017.

- [2] F. E. Casado, D. Lema, M. F. Criado, R. Iglesias, C. V. Regueiro, and S. Barro, "Concept drift detection and adaptation for federated and continual learning," *Multimedia Tools and Applications*, vol. 81, pp. 3397–3419, 2022.
- [3] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, "A framework for evaluating and benchmarking concept drift detection methods," in *Proc. KDD*, 2026, doi: 10.1145/3770855.3819070.
- [4] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, and P. B. Gibbons, "Federated learning under distributed concept drift," in *Proc. AISTATS*, PMLR 206, pp. 5834–5853, 2023.
- [5] T. Chow, U. Raza, I. Mavromatis, and A. Khan, "FLARE: Detection and mitigation of concept drift for federated learning based IoT deployments," in *Proc. IEEE IWCWC*, pp. 989–995, 2023.
- [6] I. Mavromatis, S. De Feo, and A. Khan, "FLAME: Adaptive and reactive concept drift mitigation for federated learning deployments," *arXiv:2410.01386*, 2024.
- [7] A. Patel, D. S. Tomar, R. K. Pateriya, and R. Haripriya, "FL-MalDrift: A federated learning framework for malware detection under local concept drift," *Scientific Reports*, vol. 16, Art. 1821, 2026.
- [8] Y. Han, D. Wu, and B. Varghese, "DriftGuard: Mitigating asynchronous data drift in federated learning," *arXiv:2603.18872*, 2026.
- [9] P. Fu and M. Tang, "FedDAA: Dynamic client clustering for concept drift adaptation in federated learning," *arXiv:2506.21054*, 2025.
- [10] Q. Zhou et al., "FedPLC: Federated learning with dynamic cluster adaptation for concept drift on non-IID data," *Sensors*, vol. 26, no. 1, Art. 283, 2026.
- [11] L. Baier, T. Schlör, J. Schöffner, and N. Kühl, "Detecting concept drift with neural network model uncertainty," in *Proc. 56th Hawaii Int. Conf. on System Sciences (HICSS)*, pp. 835–844, 2023.
- [12] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. ICML*, PMLR 48, pp. 1050–1059, 2016.
- [13] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. on Data Mining*, pp. 443–448, 2007.
- [14] J. L. Lobo, I. Laña, E. Osaba, and J. Del Ser, "On the connection between concept drift and uncertainty in industrial artificial intelligence," in *Proc. IEEE CAI*, pp. 171–172, 2023.
- [15] Y. Ovadia et al., "Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift," in *Proc. NeurIPS*, pp. 13991–14002, 2019.
- [16] V. Nguyen, C. Shui, V. Giri, S. Arya, A. Verma, F. Razak, and R. G. Krishnan, "Reliably detecting model failures in deployment without labels (D3M)," in *Proc. NeurIPS*, 2025.
- [17] O. Solozobov, "Label-free detection of governance evidence degradation in risk decision systems," *arXiv:2604.17836*, 2026.
- [18] L. Poenaru-Olaru, L. Cruz, A. van Deursen, and J. S. Rellermeyer, "Are concept drift detectors reliable alarming systems?" *arXiv:2211.13098*, 2022.
- [19] O. A. Mahdi, E. Pardede, S. Bevinakoppa, and N. Ali, "Federated learning under concept drift: A systematic survey of foundations, innovations, and future research directions," *Electronics*, vol. 14, no. 22, Art. 4480, 2025.
- [20] M. Böther et al., "Modyn: Data-centric machine learning pipeline orchestration," *Proc. ACM on Management of Data*, vol. 3, no. 1, Art. 55, 2025.
- [21] M. F. Criado, F. E. Casado, R. Iglesias, C. V. Regueiro, and S. Barro, "Non-IID data and continual learning processes in federated learning: A long road ahead," *Information Fusion*, vol. 88, pp. 263–280, 2022.
- [22] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. ICLR*, 2019.
- [23] K. Panchal et al., "Flash: Concept drift adaptation in federated learning," in *Proc. ICML*, PMLR 202, pp. 26931–26962, 2023.
- [24] X. Chen et al., "Virtual concept drift detection and adaptation in federated data stream learning," *Int. J. Data Science and Analytics*, vol. 21, Art. 46, 2026.
- [25] M. Stallmann, A. Wilbik, and G. Weiß, "Towards unsupervised sudden data drift detection in federated learning with fuzzy clustering," in *Proc. IEEE FUZZ-IEEE*, 2024.
- [26] R. A. Bhope, K. R. Jayaram, P. Venkateswaran, and N. Venkatasubramanian, "Shift happens: Mixture of experts based continual adaptation in federated learning," in *Proc. ACM Middleware*, pp. 455–468, 2025.
- [27] G. Canonaco, A. Bergamasco, A. Mongelluzzo, and M. Roveri, "Adaptive federated learning in presence of concept drift," in *Proc. IEEE Int. Joint Conf. on Neural Networks (IJCNN)*, pp. 1–7, 2021.
- [28] P. Lu, J. Lu, A. Liu, and G. Zhang, "Early concept drift detection via prediction uncertainty," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 39, no. 18, pp. 19124–19132, 2025.
- [29] B. Zhang, H. Guan, H. K. Lee, R. Liu, J. Zou, and L. Xiong, "FedSGT: Exact federated unlearning via sequential group-based training," *arXiv:2511.23393*, 2025.